Parallel Optimization of JPEG Compression Algorithm

Team 6

311551170 林琨堯 312553008 陳沛圻 312553027 林煦

Contents.



Problem statement

Approaches & Results

Conclusion

RGB to YCbCr

DCT

Quantization

ZigZag

run-length encoding

Huffman Encoding

Compression

Problem statement

JPEG Compression

Decompression

Huffman Decoding

YCbCr to RGB

IDCT

Dequantization

Inverse ZigZag

Run-length Decoding

DCT.

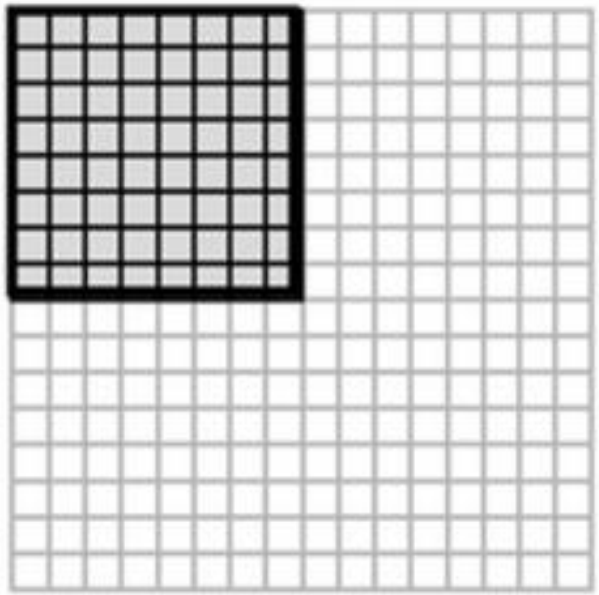
將影像的 Y、Cb 和 Cr 分量，以每個 8×8 區塊為單位，轉換為頻率數據。

DCT 計算公式：

$$F(u,v) = \frac{1}{4}\sum_{x=0}^{7}\sum_{y=0}^{7} f(x,y)\cos\left[\frac{(2x+1)u\pi}{16}\right]\cos\left[\frac{(2y+1)v\pi}{16}\right]$$

$f(x,y)$：區塊內的像素值

$F(\mathrm{u},v)$ ：頻率係數



Group 8x8 pixels block

Quantization.

將 DCT 系數 除以量化矩陣，減少精度並捨棄次要數據。

Quantization 計算公式：

$$Q(u,v) = round(\frac{F(u,v)}{Q_{table}(u,v)})$$

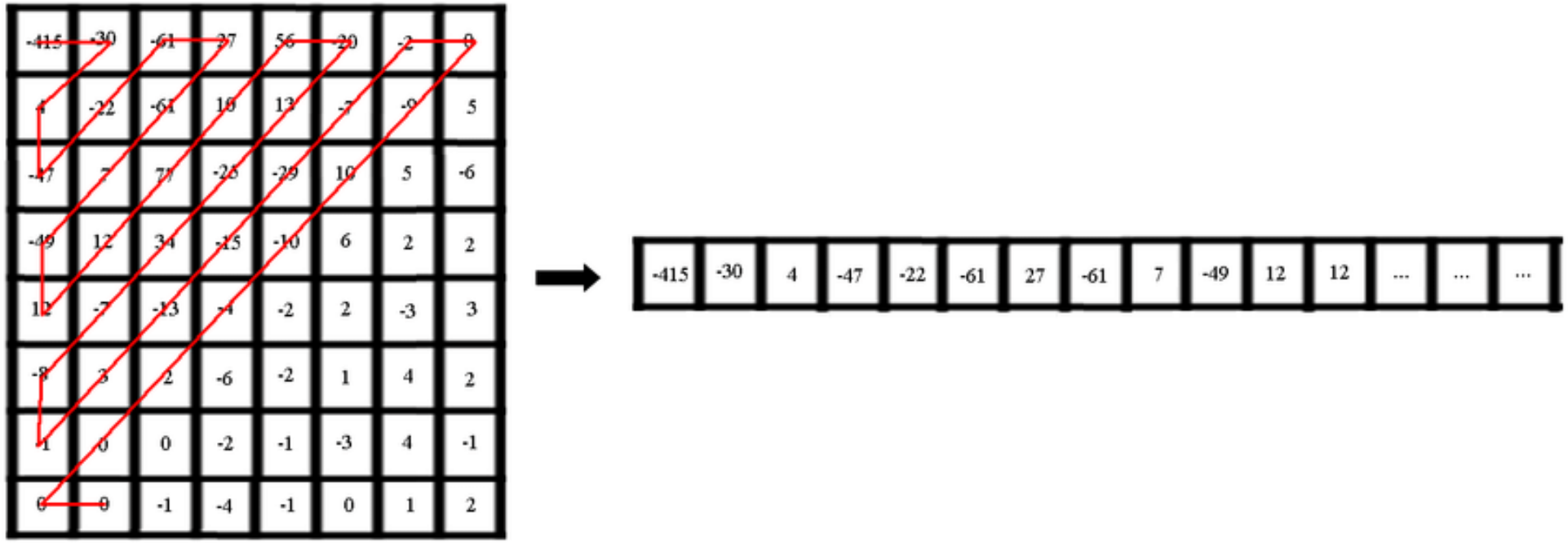
$F(\mathrm{u},v)$ ：**DCT** 係數

$Q_{table}(u,v)$ ：量化表中的對應值

$\mathbf{Q}(u,v)$ ：量化後的值

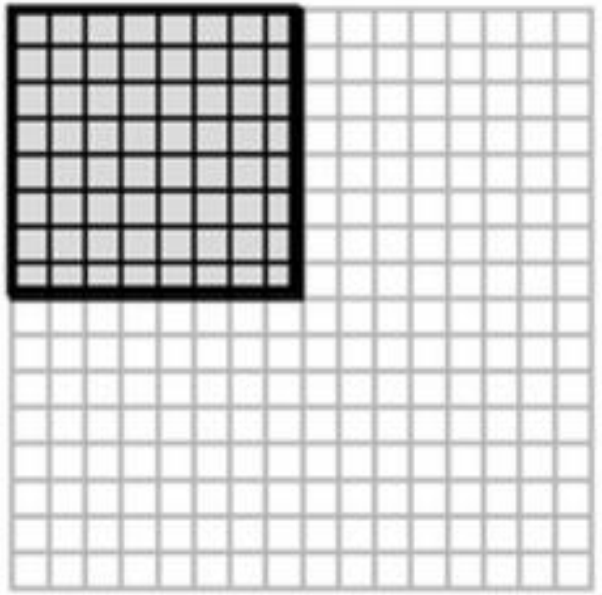
Zigzag.

將量化後的 8×8 DCT 系數轉換為一維陣列。
低頻成分排在前方，高頻成分（多為 0）排在後方。



Problem statement.

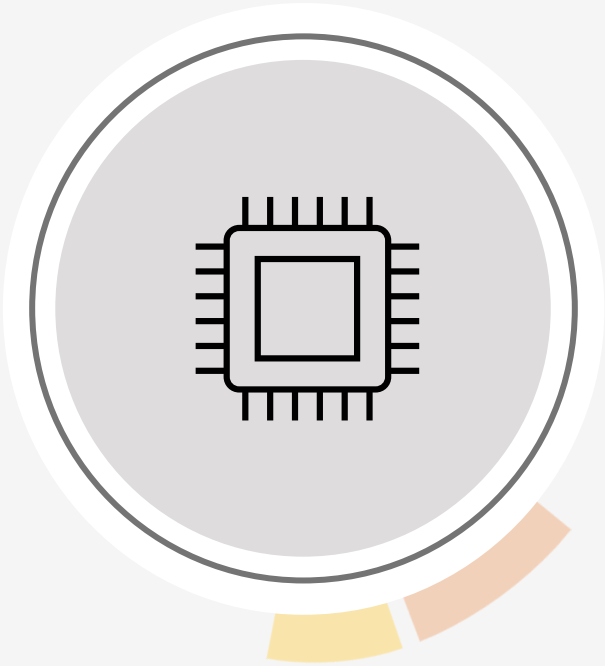
Approximately overall Complexity: $O(M * N)$



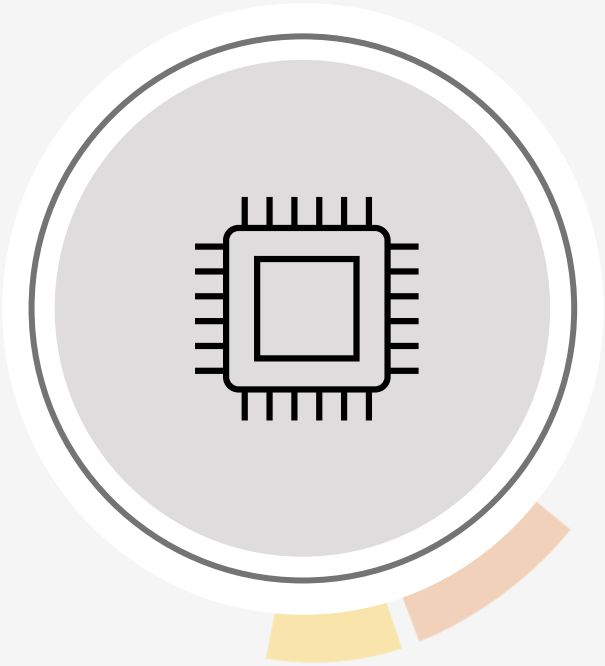
Group 8x8 pixels block

Approaches.

Pthreads



OpenMP

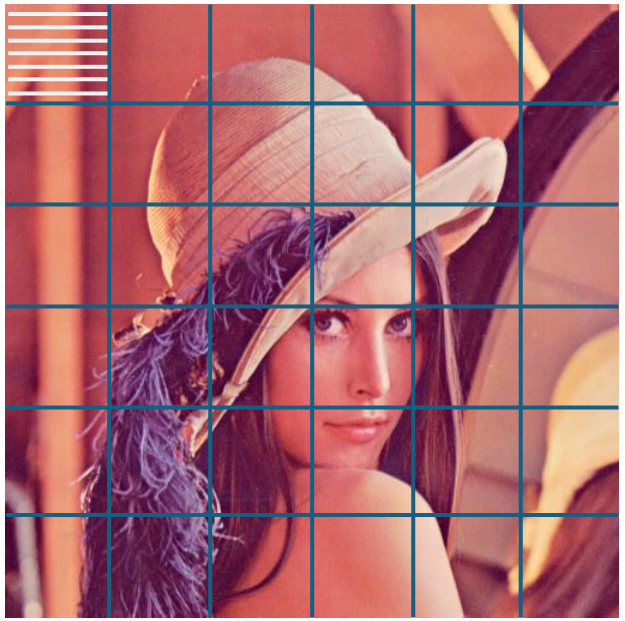


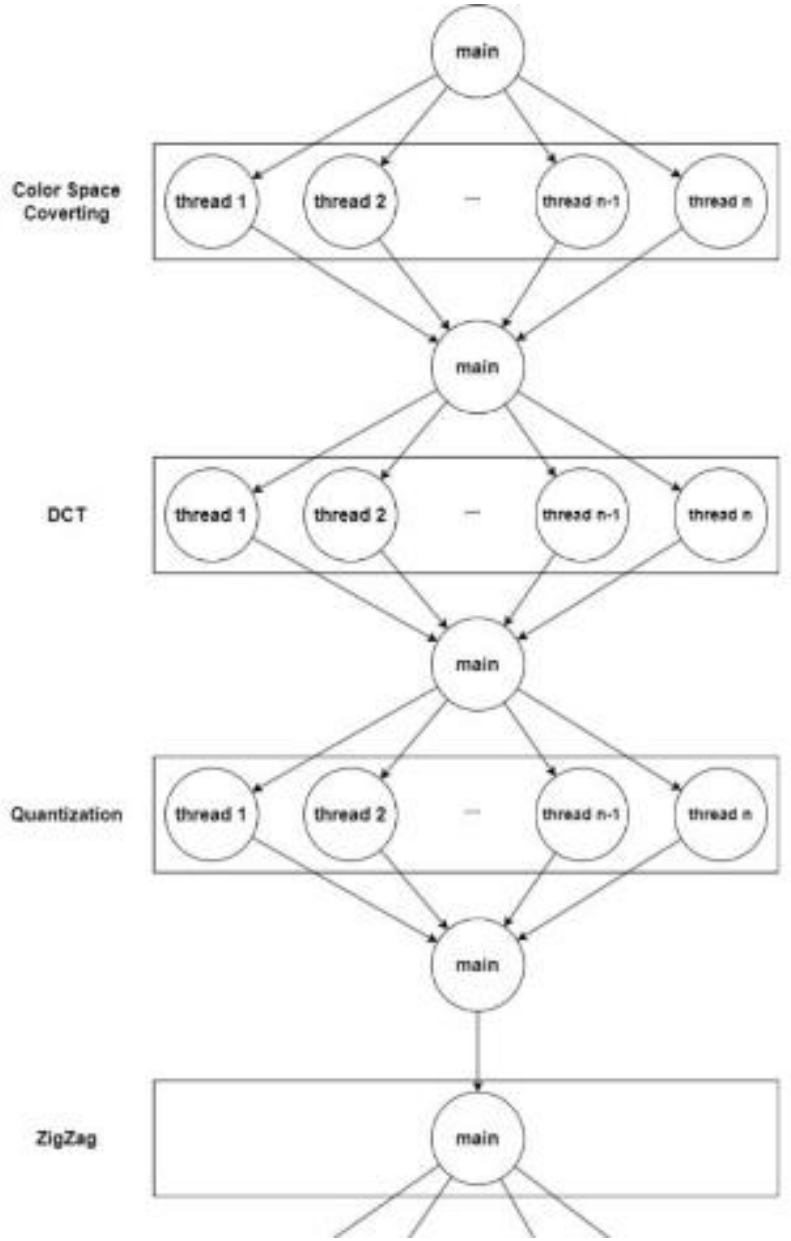
CUDA



Pthread & OpenMP

1. 對各個block平行化 :
   Each thread contains
   (8 / 6 + 8 % 6) rows

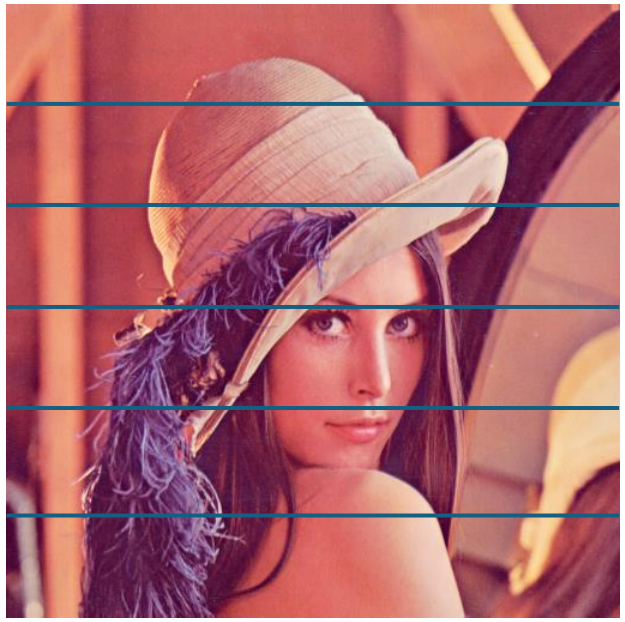




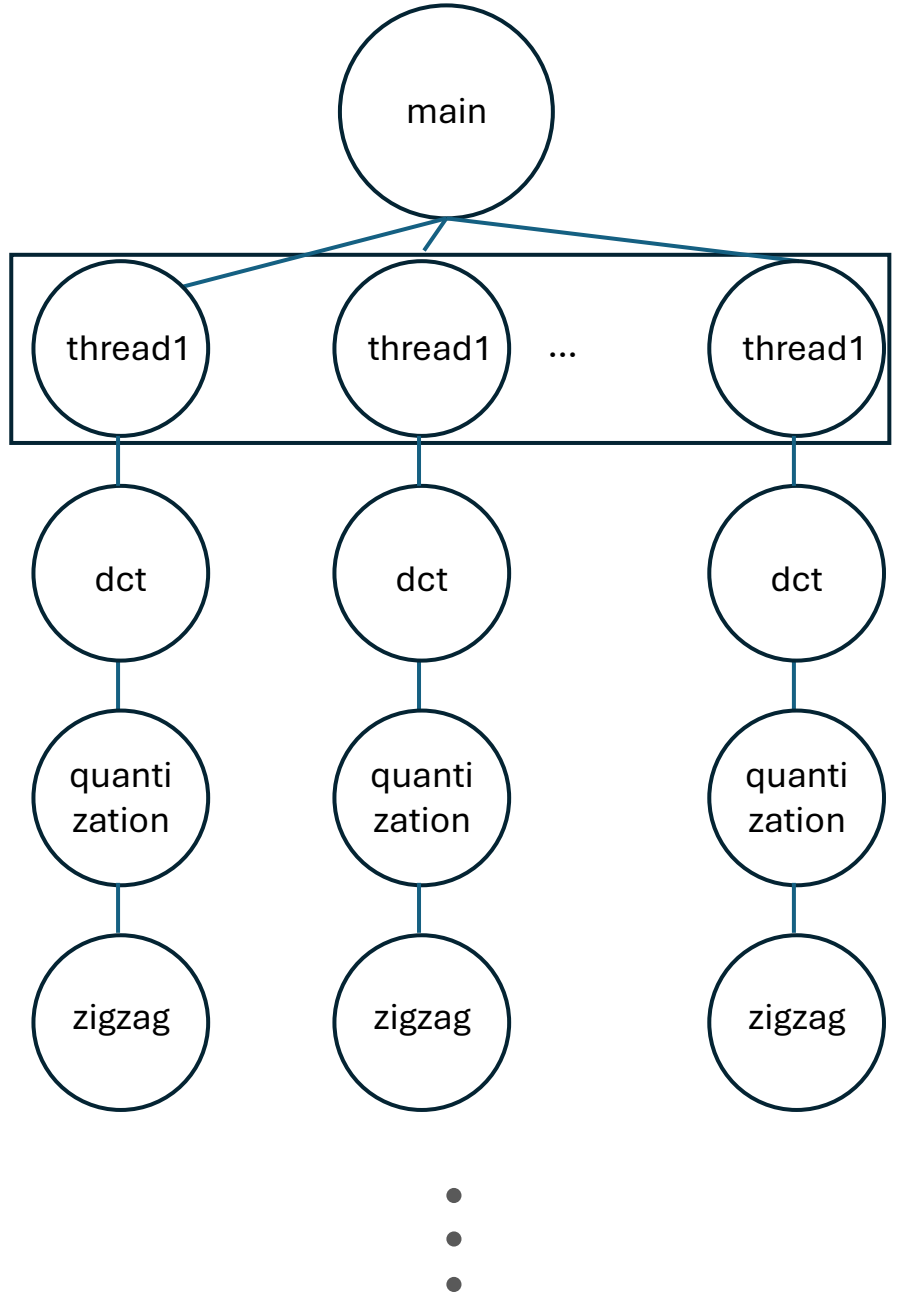
RLE and Huffman 採用 serial

Pthread & OpenMP

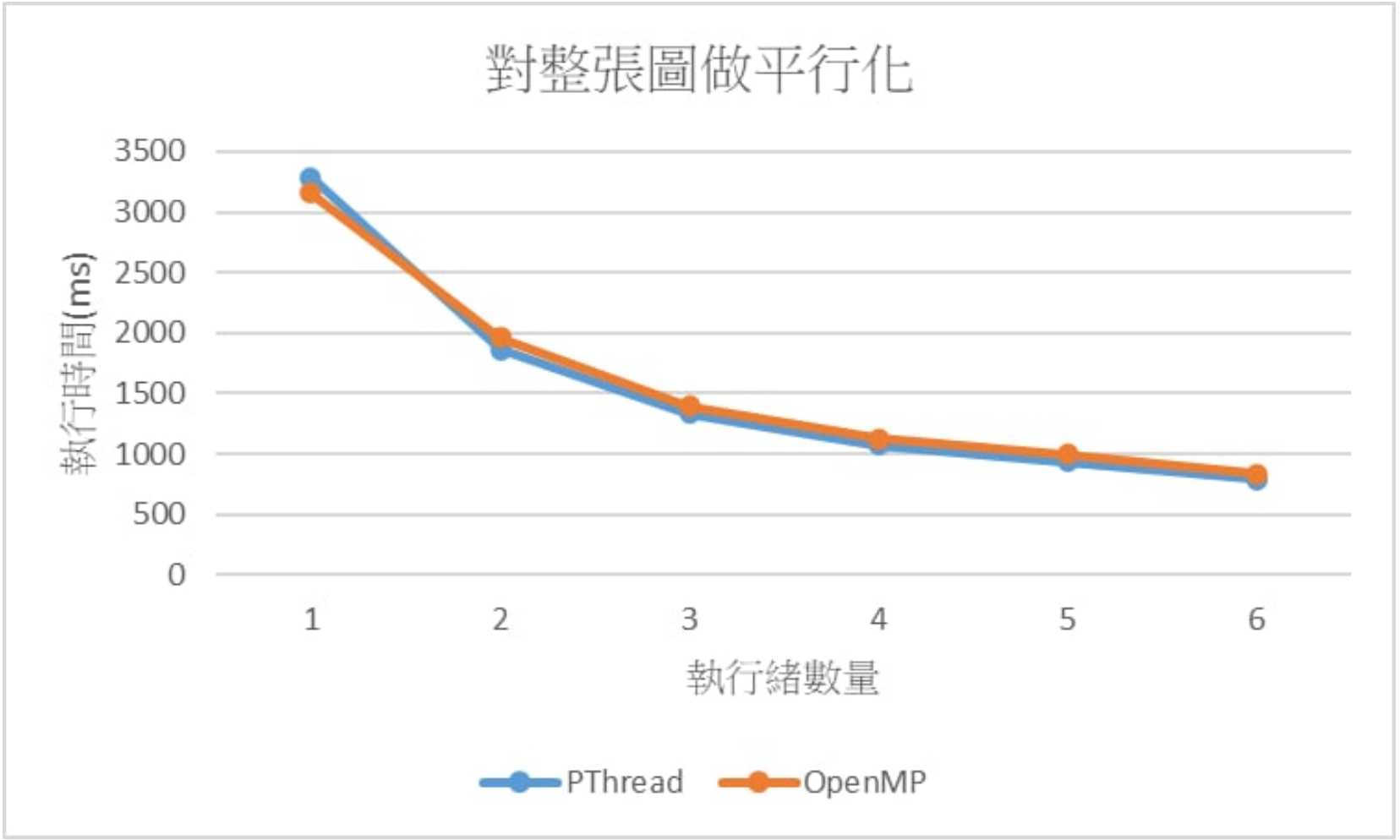
2. 對整張圖平行化
Each thread contains
(height / 6 + height % 6) rows



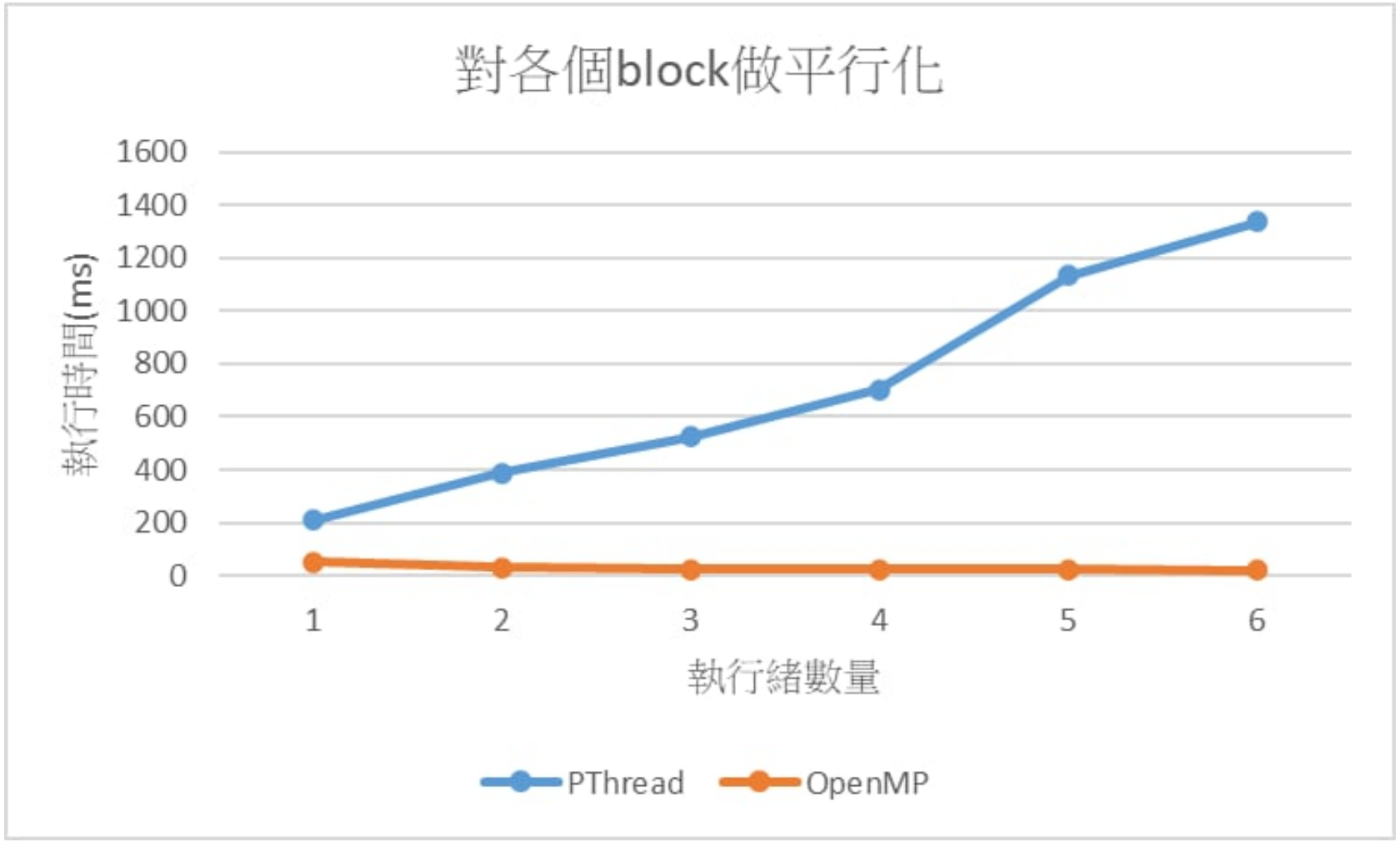
Image



Pthread & OpenMP Results.

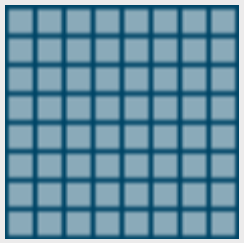


Pthread & OpenMP Results.

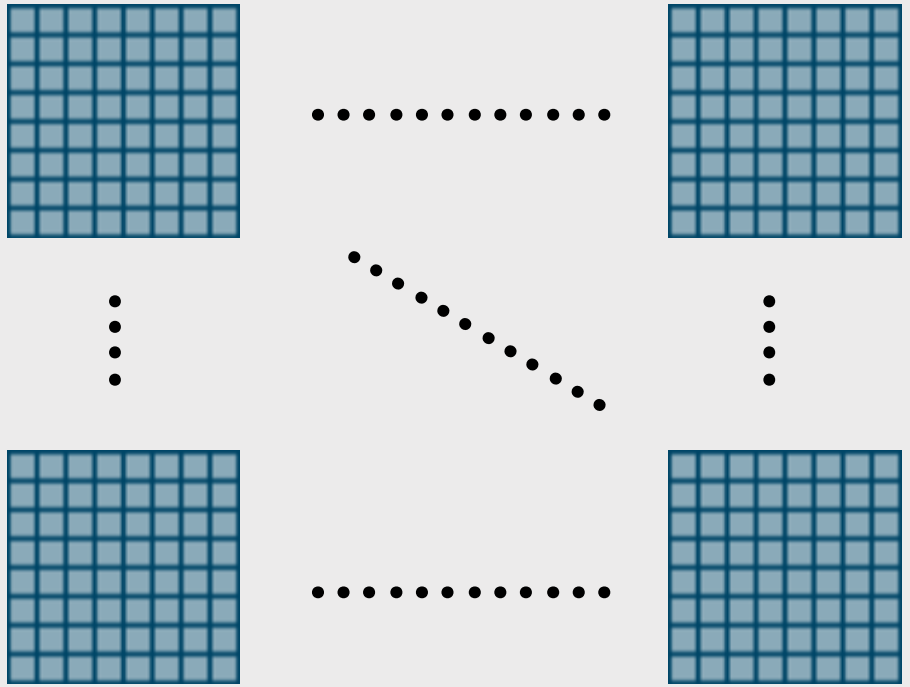


CUDA.

1. Per 8*8 block 進行加速



2. Entire Image 進行加速

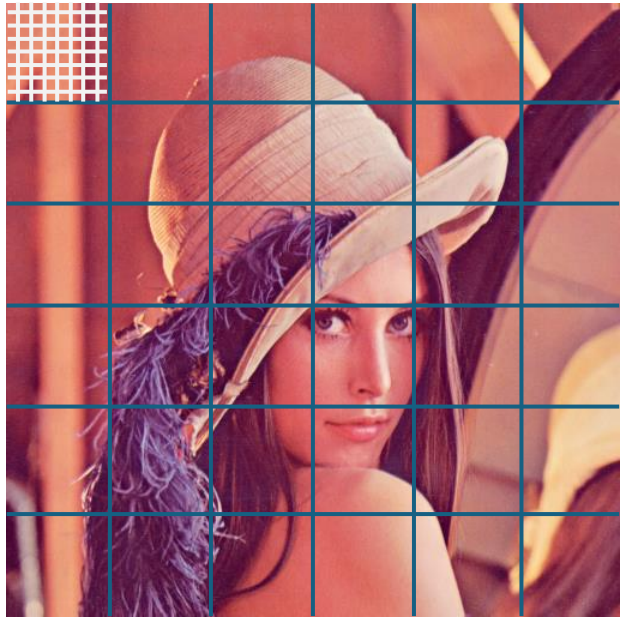


CUDA proposed solution.

- 第一種方法會因為 CPU -> GPU、GPU -> CPU 的次數太多而導致 overhead 過高，進而導致效率低於 serial 的版本
- 為了要解決 Overhead 過高的問題，將整個圖片進行平行化，結果就大大降低了整體的時間秒數

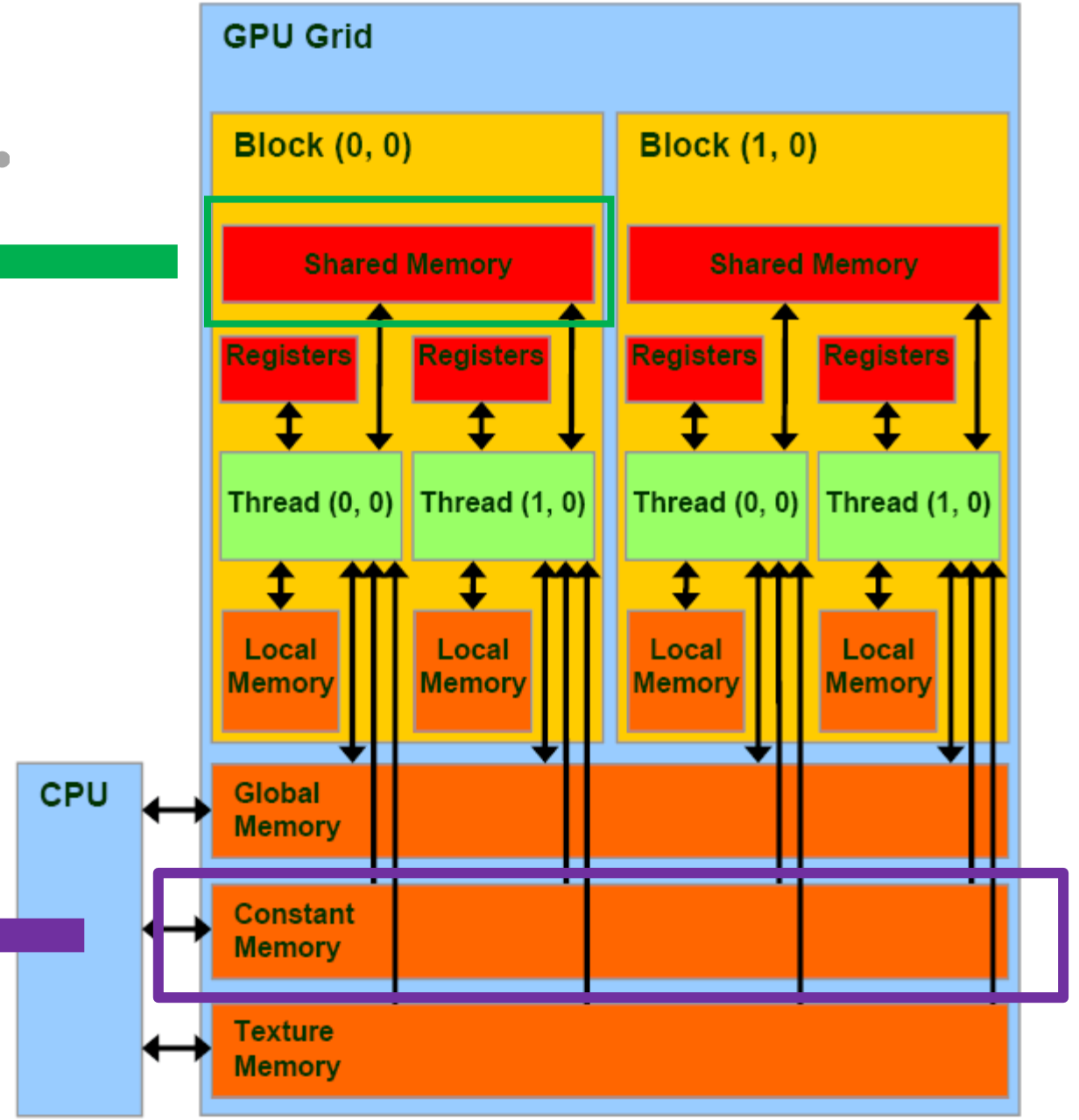
<<threadX, threadY>>
Threads (white)

<<blockX, blockY>>
Grid Blocks (blue)



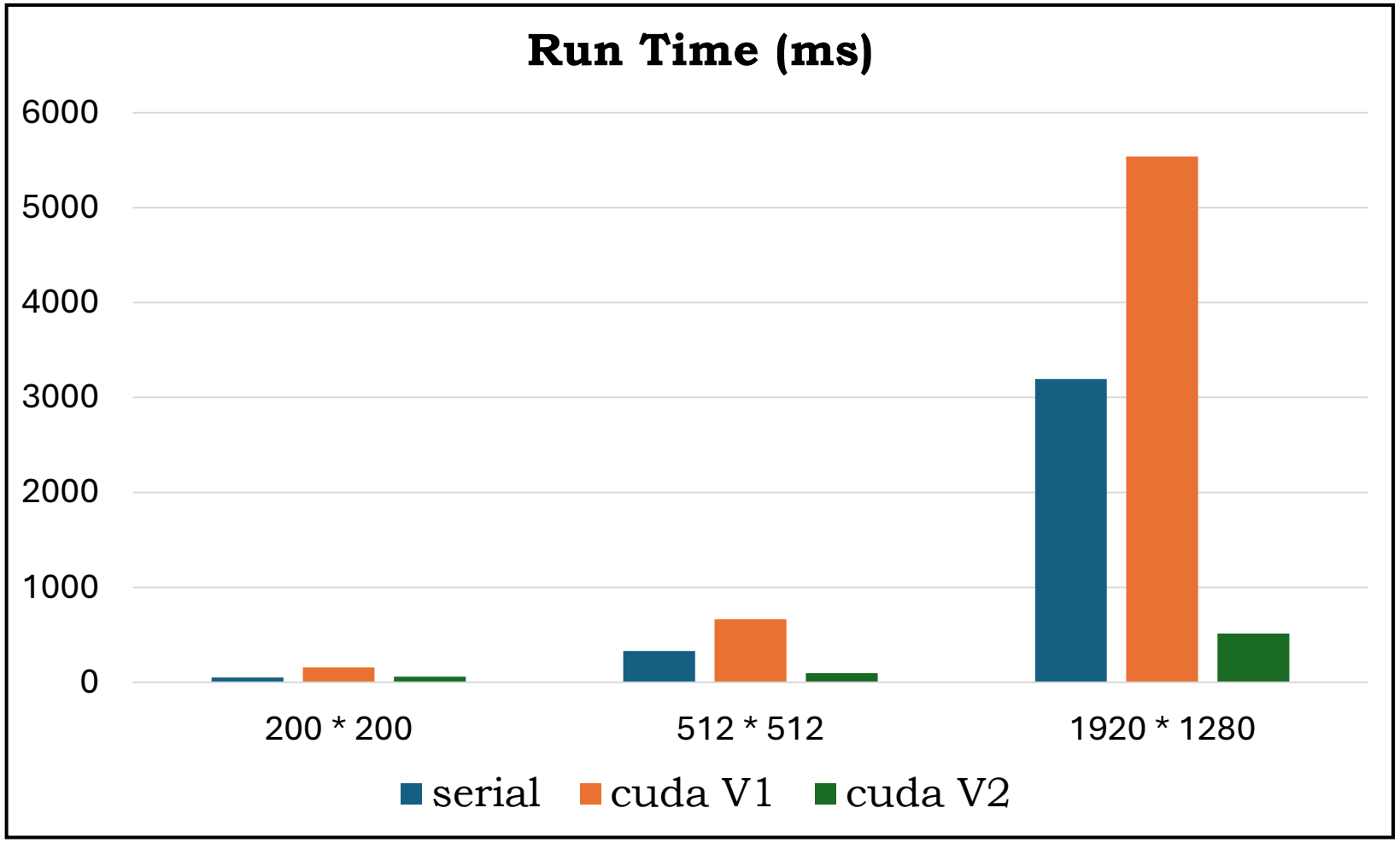
CUDA Details.

8 * 8 pixels colors

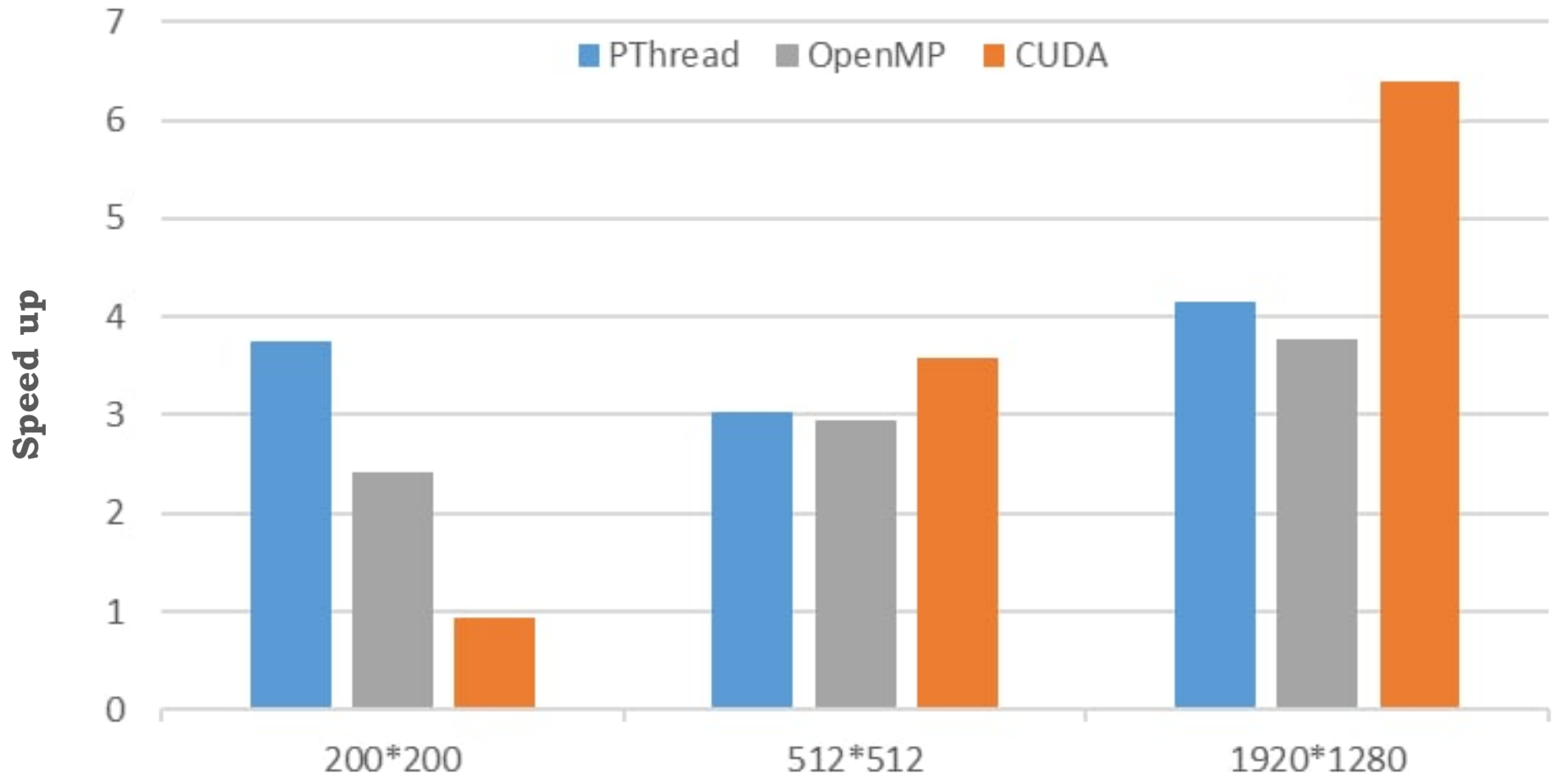


1. precomputedCosine
2. zigzagIndices
3. Chrominance quantization table
4. Luminance quantization table

CUDA Results.

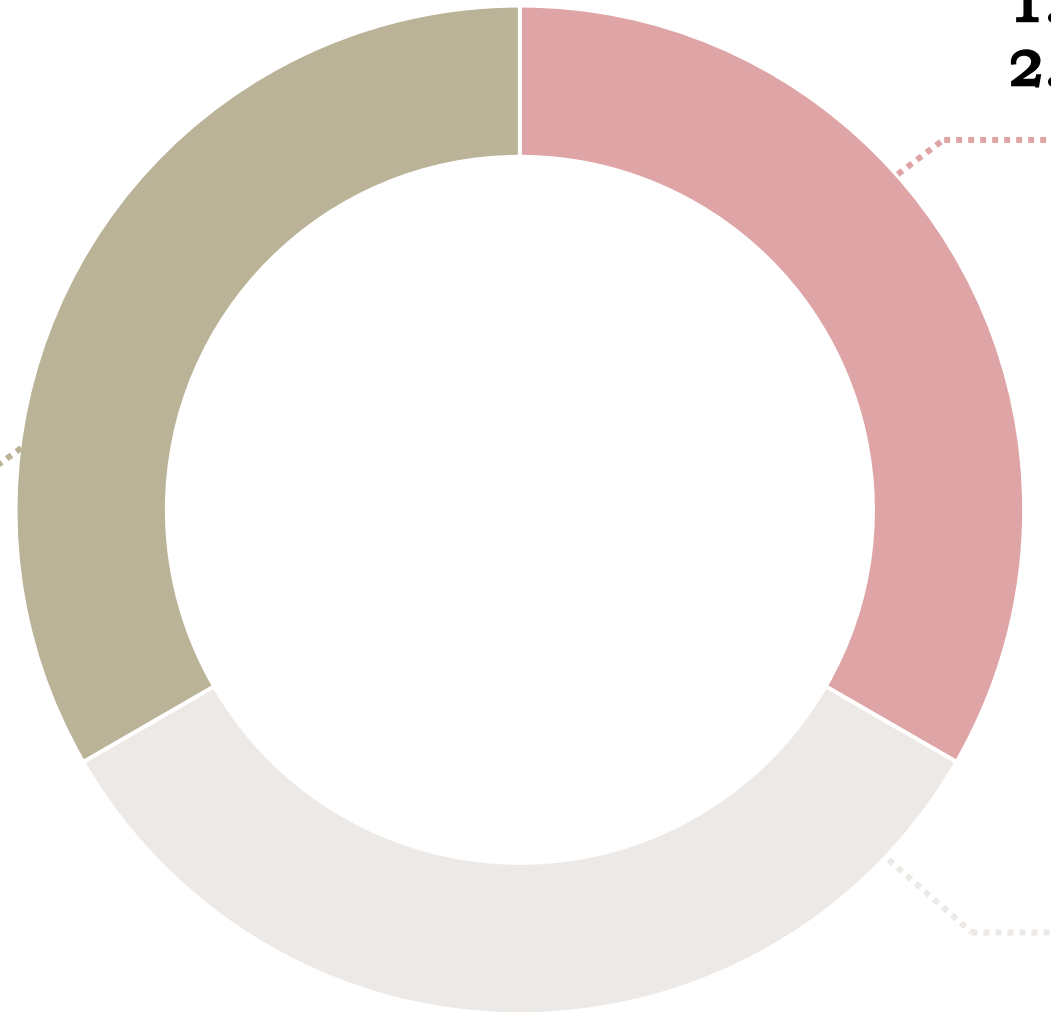


All Model Comparison.



Work distribution.

陳沛圻
1. **Serial code** 程式實作
2. **Cuda** 程式實作



林琨堯
1. **Serial code** 程式實作
2. **Pthreads** 程式實作

林煦
1. **Serial code** 程式實作
2. **OpenMP** 程式實作

Conclusion.

1. **Pthread** 和 **OpenMP** 的加速撰寫邏輯大致相同，但 **Pthread** 需要手動管理執行緒的創建與同步，若管理不當，可能導致效率未能有效提升。

2. 在實驗不同的平行化方式時，我們發現資料讀取是影響平行化速度的關鍵之一，尤其在頻繁存取或大規模數據處理時，會直接影響整體運算性能。

3. 針對 **CUDA**，處理較小圖像時的加速效果不如其他方法，甚至低於序列化執行（**serial**）。推測原因可能在於大量時間消耗在 **CPU** 與 **GPU** 間的資料傳輸上，導致總執行時間增加。

Q&A

Thank you.